

Q: Any questions or
comments?

Lecture 18: Failure and Reliability

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

- Failures
- Coping with failure
- Networking and Reliability
 - Networking issues for distributed systems
 - Reliable one-to-one communication

Failures (Section)

Failure Models

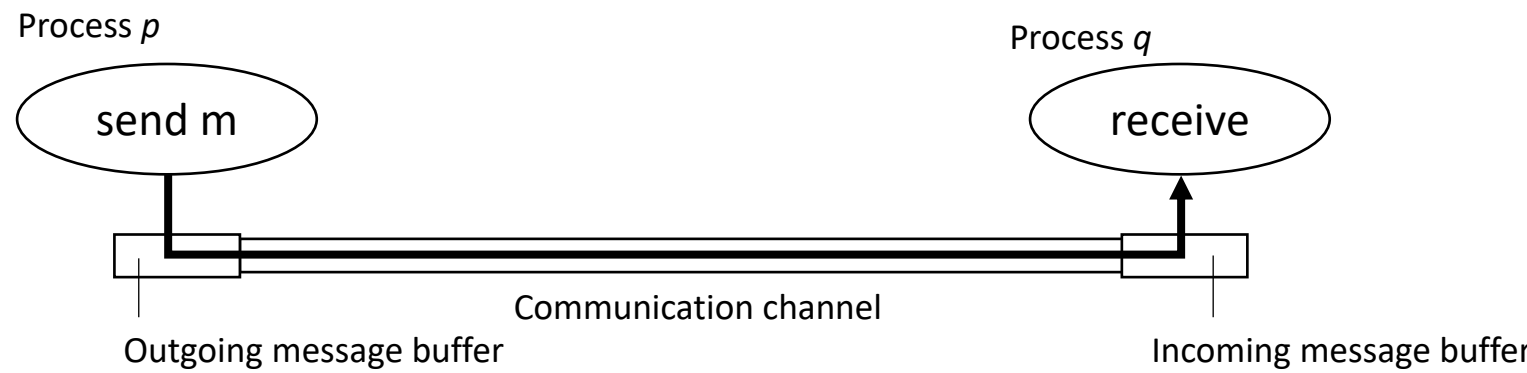
- Failure models deal with:
 - how (parts of) a distributed system can **fail** (this section); and
 - what can be done to **detect** and **mask** such **failures** (next section)
- Types of failure include (see following slides):
 - **Omission failures** – not doing something you should
 - Process or Communication
 - **Arbitrary failures** – any other failure 😊
 - **Timing Failures** – not doing something in time



Process omission failures

- i.e. when a **process** fails to perform an action it is supposed to
- Most commonly the process may **crash**
 - i.e. cease to perform any further actions at all
- Other processes may detect that the process has crashed using **timeouts**, e.g. if it repeatedly fails to respond
- A process crash is “**fail-stop**” if other processes can detect with certainty that it has failed
 - But this is only possible (remotely) in a synchronous system
 - In an asynchronous system messages showing it is alive might be lost or still delayed (see earlier lecture)

Communication omission failures



- Consider a simple message communication:
 - Process p performs a **send** operation on a message m by inserting it into an outgoing message buffer
 - The communication channel transports message m to process q 's incoming message buffer
 - Process q performs a **receive** operation to take message m from its incoming message buffer and delivering it
- The message can be lost at each stage
 - e.g. due to congestion in the network, lack of space in a buffer or corruption of the message

Arbitrary failures

- **Arbitrary** or **Byzantine** failures describe the worst possible failure semantics, in which any type of error may occur:
- A **process** may omit arbitrary steps and/or insert arbitrary steps
 - E.g. due to bugs in the code, or due to the presence of malicious processes
- A **communication channel** may omit messages, insert arbitrary messages and/or make arbitrary change to messages
 - But these are less common observed, as (for example) the Internet Protocols include some checks/mitigations for this

Timing failures

- In a synchronous system time limits may be set on the performance of certain tasks
 - E.g. completing a computation or responding to a request
- A **timing failure** is a failure to perform this task within the specified time limit
- To create a synchronous distributed system requires **real-time** operating systems and networks with high-reliability and performance guarantees
 - This is typically more complex and expensive than an asynchronous / non-real time system (see also Interaction Models)

Q: How can a distributed system cope with failure?
For example??

Coping with Failure (Section)

Masking and mitigating failures

- Each component in a distributed system is typically constructed from a collection of other components
- If part of the system can **detect** the failure then it may be able to **mask** the failure, either concealing it entirely or converting to a more acceptable type of failure
- => A reliable system can be constructed from unreliable components (if there are enough of them)

Failure Masking Examples

- **multiple** servers holding copies of the same data can continue to provide a service when one of the servers has crashed
- message **checksums** detect corrupt packets, which are discarded, converting an arbitrary failure to an omission failure
- array bounds **exceptions** detect illegal memory accesses (which would give arbitrary results, i.e. Byzantine failure) and terminate the task/program (allowing a clean stop, i.e. omission failure, or recovery actions)

Q: is a reliable system (free from failures) also safe? :)

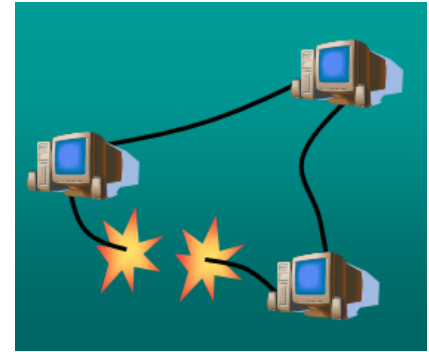
Fault tolerance (and associated terminology)

- **Failures** def.
 - "when the behaviour of a system deviates from that which is specified for it" / "deviation from desired or intended state"
- **Errors**
 - internal discrepancy between intended and actual behaviour, caused by a fault
- **Faults** def.
 - A defect in the system, e.g. bug, lost message
- **Fault tolerance**
 - "the ability to operate correctly [i.e. without failure!] in the presence of faults"
 - (but necessarily with some degradation, e.g. lower performance or resilience)

Failure detection

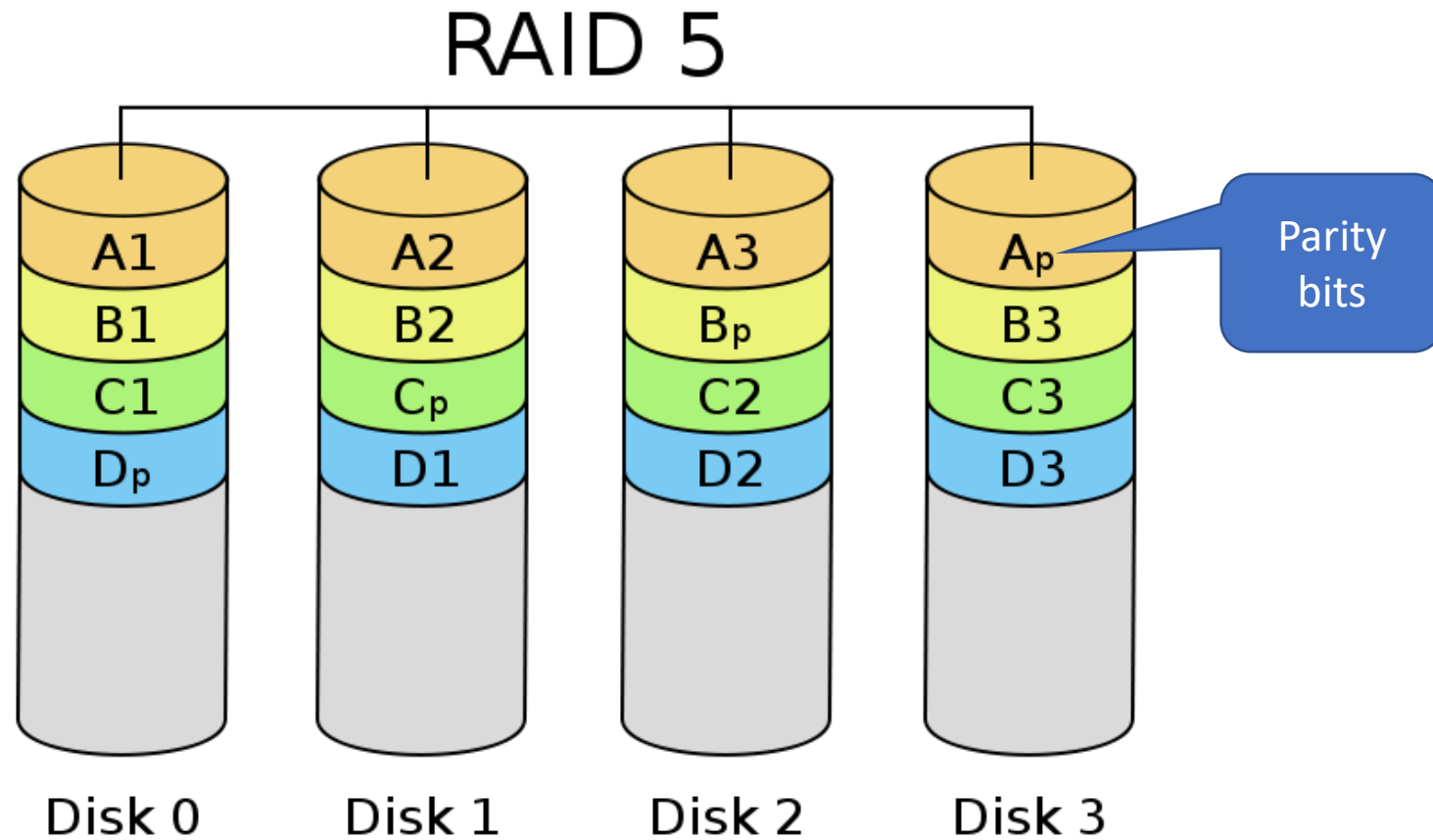
- Typically a process will need to **detect** a failure (or error) in another process or communication channel before it can respond to it
- Different failures are detected in different ways, e.g.
 - Checksums for corrupted data/messages;
 - Sequence numbers for non-existent or duplicate messages;
 - Timestamps and timeouts for lost messages or crashed processes
- But as some failures cannot be detected with certainty in an asynchronous system
 - E.g. timeouts are not definite checks – you're not sure if a web server has really crashed or (e.g.) the network is slow
 - So asynchronous systems must also cope with false alerts of failures

Achieving fault tolerance



- The system might try to detect a fault then **re-try** the failed work
 - E.g. resend a message, send the request to a back-up server
- And/or **extra** work can be done from the outset “just in case” some of it will fail
 - E.g. error-correcting codes, multiple copies of messages,
 - multiple servers doing the same work at the same time,
- => require **redundancy**, i.e. additional resources/work beyond that needed with no faults

Example: RAID-5



Data from any N-1 disks is needed
If one disk fails data is OK
(but cannot cope with a second failure)

Replication

- **Replication** = having [entire] extra 'copies' of data and/or processes
 - One (not the only) form of redundancy
- for:
 - **Fault tolerance** - above
 - Including **High Availability**
 - i.e. the percentage of time that a service is available
 - E.g. coping with server crashes or network partition
 - But also e.g. disconnected operation (client-side replication)
 - **Performance**
 - E.g. cacheing in DNS and WWW, content distribution networks

Networks and Reliability (Section)

Networking issues for distributed systems (1)

- Distributed systems by definition involve networks, and the characteristics of those networks impact the distributed system, in particular:
- **Performance**: the speed at which messages can be transferred depends on the **latency** and **bandwidth** of the underlying networks
 - Round-trip latencies on a LAN can be about 0.5ms (1/2000th second); on the Internet they can be 5-500ms (typically 20-200ms), i.e. 10-1000 times slower
 - (reliable) Data transfers take longer on lower-bandwidth networks

Networking issues for distributed systems (2)

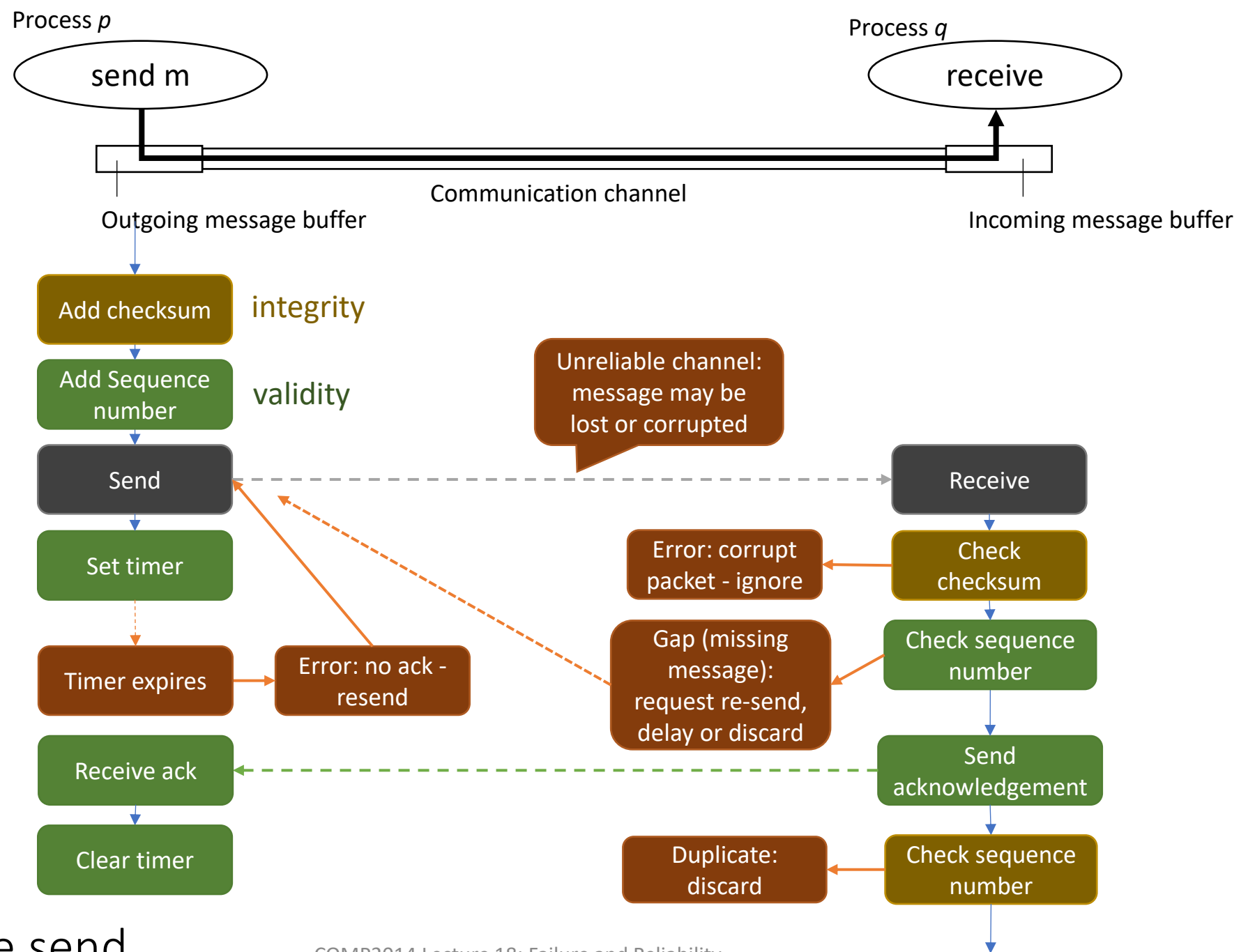
- **Scalability:** if the network (e.g. the Internet) does not scale then the distributed system cannot scale. Fortunately the Internet is scaling quite well, but work is ongoing to improve this (e.g. IP version 6)
- **Reliability:** reliable systems can be built from unreliable parts. But unreliable networks still typically limit performance (e.g. taking time to recover).
- **Mobility:** the Internet has only limited support for devices (e.g. phones) moving between networks, so additional support can be needed from the distributed system.

Networking issues for distributed systems (3)

- **Quality of Service:** to make QoS guarantees typically requires that the network can also make such guarantees, which many networks cannot.
- **Multicast:** applications can simulate multicast (one-to-many communication) to some extent, but when provided by networks it can increase (local) efficiency and reliability and reduce configuration

Reliable one-to-one communication

- A **reliable communication service** can be built over an unreliable channel
 - E.g. TCP/IP over the Internet (IP)
- Reliable communication is characterised by:
 - **Validity**: any outgoing message is eventually delivered
 - Typically this will involve **retransmitting** lost or corrupted messages
 - **Integrity**: the message received is the same as the one sent, and no messages are delivered twice
 - Typically this will involve adding information (**checksums**) to detect message corruption
 - and adding **sequence numbers** to ensure that retransmitted messages are not delivered twice



Reliable message send

Failures Summary

- **Failure models** deal with how (parts of) a distributed system can fail and what can be done to detect and mask such failures
- A **process omission failure** is typically where a process crashes.
 - In a synchronous distributed system this can be detected => fail-stop
- A **communication omission failure** is resulting in non-delivery of a message
- **Arbitrary** or **byzantine** failures include all possible errors and omissions
- A **timing failure** is where a task is not performed within a specified time (in a synchronous distributed system)

Coping with Failure Summary

- **Fault tolerance** is the ability to provide correct service in the presence of (internal) **faults** (errors)
- Failures can potentially be **detected**, e.g. by checksums, timeouts
- Failures can potentially be **masked** or at least converted to more acceptable failure types, e.g. discarding corrupt messages
- Fault tolerance generally requires some kind of **redundancy**, i.e. additional resources/work beyond that needed with no faults
- E.g. **Replication** (of data or processes) can provide fault tolerance and also **performance** and **availability**

Networks and Reliability Summary

- The underlying network characteristics will have knock-on effects on the distributed systems that use them, e.g. in **performance, scalability, reliability, mobility, quality of service** and **multicast**.
- A **reliable communication service** can be built over an unreliable channel. Typically this uses
 - **checksums** to detect message corruption,
 - **sequence numbers** to detect duplication or reordering and
 - **acknowledgments** plus **timeouts** and **retransmission** to recover from lost or corrupt messages.

Next

- (Easter :)
- Lecture 19: Data and Failure
 - i.e. more about coping with failure, in particular 2-phase commit, data replication
- Lecture 20 & 21: Q&A / revision
- (final) Drop-in labs